



A Fast Algorithm for High-Dimensional Causal Discovery

Sakib A. Mondal¹ · Prashanth Rv¹ · Sagar Rao¹

Received: 13 October 2024 / Accepted: 13 May 2025 / Published online: 8 June 2025
© The Author(s), under exclusive licence to Springer Nature Switzerland AG 2025

Abstract

Discovering causal relationships from observed data has applications in many fields including Bioinformatics. The PC (Peter and Clarke) algorithm and its parallel version are the state-of-the-art methods for causal discovery. However, the high runtime of these algorithms makes them inefficient for high-dimensional data. For example, to discover causal regulatory expression in gene, the number of nodes (factors) can easily be more than 2000. For such an instance, even the fastest variant of PC today would take days to complete. To solve this, we propose a recursive parallel causal discovery algorithm (RPCD). RPCD constructs the graph recursively, which leads to significant savings in the number of conditional tests. We evaluated RPCD on a number of causal network datasets as well as on a real-life incident dataset. The experimental results show that it improves both the running time and accuracy metrics significantly.

Keywords Causal discovery · Gene expression · Parallel PC

1 Introduction

Study of the association of various variables and interpretation of their relationships have been an interesting research topic for a long time. For example, in the case of gene expression, often the objective is to identify regulator genes from a set of associated genes that appear together. In the case of software incidents, a number of anomalous events from various subsystems can co-occur. However, a cause-and-effect relationship between these events is helpful to identify the root cause.

Causal inference has been studied by many scientists over the years [32–34, 40]. It has been observed through a number of experiments that causal relationships can be discovered from observational data [15, 34, 39]. A possible approach is through a randomized controlled trial. To study if a specific gene X effects on other target genes, two groups are created: one control and another effect. Gene X is fixed in the control group only. If there is a significant level of change in the

target gene's expression between the two groups, it can be concluded that gene X affects the target gene. The problem with this approach is humongous cost of the experiment.

Hence, alternative approaches have been proposed. The causal Bayesian network, i.e. CBN [35], is a prominent method. In CBN, causal relationships are captured as a directed acyclic graph, also known as DAG, DAG where variables as nodes and relationships as edges. Inferring a DAG is a non-trivial task. Two approaches have been proposed in the literature: search and score, and constraint-based. The first approach is NP-hard. The second approach has exponential complexity (in cardinality of nodes).

Peter and Clark proposed the PC algorithm [40] based on Inductive Causation, i.e. IC [43]. It constructs an undirected clique graph. Subsequently, edges are deleted using conditional independence. The algorithm is widely used in Bioinformatics [20, 28, 45, 46]. A number of open-source software based on the algorithm TETRAD [40], pcalg [19] and bnlearn [36] are used today as reliable tools for causal exploration.

One of the drawbacks of the algorithm is its dependence on the evaluation order of variables. If the order in which the variables are evaluated changes, the outcome will differ. This leads to the question of the reliability and stability of the causal relations discovered. Colombo et al. [6] discovered that a majority of the edges (40%) discovered in gene expression are not stable. This was due to order-dependence:

✉ Sakib A. Mondal
abdulsakib.mondal@walmart.com

Prashanth Rv
Prashanth.Rv@walmart.com

Sagar Rao
sagarrao@google.com

¹ Walmart Global Tech, Building 10, SEZ Cessna Business Park, Outer Ring Rd, 560087 Bengaluru, Karnataka, India

the output of the algorithm depends on the order in which edges are considered. To overcome this drawback, several variants have been proposed [6, 9, 40]. Colombo et al. [6] proposed stable-PC algorithm. It fixes neighbours only after running all the conditional independence (CI) tests at a given level. Obviously, it increases the number of CI tests, making the algorithm slow.

As such, the worst-case running complexity of the PC algorithm is exponential in number of nodes. This coupled with order-dependence, makes the algorithm a bad choice for large-scale graph learning. The parallel-PC algorithm [21] has been proposed to alleviate this problem. However, it still does not live up to the promise of large-scale discovery. For example, in an incident diagnostic domain, if there are more than 2000 variables, the parallel-PC algorithm could not give any results even after 24 h of running.

We propose a recursive faster discovery algorithm. As in parallel PC, the tests at a level are aggregated in groups, and tests from different groups can be run in parallel. Results from the tests are aggregated at each level. There is also parallelism at higher levels. We recursively build the causal structure of the whole system based on bottom-up recursive merging of components and structural inference.

This paper makes the following contributions:

1. We propose a fast and scalable algorithm (RPCD) capable of handling datasets with high dimension. Besides, it is stable, i.e. results do not depend on the order of evaluation of the variables.
2. Through experiments, we demonstrate the efficacy of the RPCD algorithm in two real-world applications—incident diagnostics and gene expression discovery. Additional experiments on a set of benchmark instances too establish superiority of the RPCD algorithm.

We have organized the paper as below. Section 2 covers the related work. The PC algorithm and its stable and parallel variants are presented in this section. The RPCD algorithm is covered in Sect. 3. Experimental results are presented in Sect. 4. Concluding remarks and discussions are provided in Sect. 5.

2 Literature

Under certain assumptions, Bayesian networks can be thought of as causal networks [16, 34]. Therefore, Bayesian networks or probabilistic graphical models (PGM) have been used as causal discovery tools. In PGM, nodes represent variables and edges represent pairwise relationships among the nodes. The objective of causal discovery is to derive a DAG capturing the dependencies among the variables [10].

A class of causal discovery methods [8, 14, 16] find a DAG with best fitment to the data based on a score. This is a NP-hard problem [5].

Another class of methods [7, 34, 38] eliminate edges when there is no causal relationship between the variables corresponding to the end-points of the edge. Despite its exponential complexity, its performance is satisfactory in practice, where the datasets are often sparse [18]. The PC algorithm [40] is a notable solution. However, it has high running time and inaccuracy. Several proposals aimed at improving on these aspects. Steck and Tresp [41] reduce the CI tests to improve the overall running time. This was based on the following observation: to check for edges between nodes X and Y , conditioning on nodes outside the $X - Y$ trail is unnecessary. This reduces the number of CI tests, but increases time to find the nodes not in the trail. Therefore, there is no significant improvement in running time, contrary to the objective.

Several proposals [1, 7, 29] aimed at improving the accuracy through improved CI tests. (Chi-square tests were replaced with Bayesian tests.) But, it is still not evident whether the replacement tests improve the performance in real-life datasets. Besides, most of these approaches concentrate on the parallelization of only the CI tests rather than a holistic view of the whole problem. [25] proposed mutual information-based features instead of CI to reduce the computation without the degradation of accuracy.

Other techniques from PGM have been used to devise alternative constraint-based methods of learning causal graphs. Cooper [7] and Silverstein et al. [38] proposed methods to search for only three sets of variables at a time and build the graph incrementally. In Cooper [7], three variables P , Q and R are analysed for pairwise correlation. If P and R are independent conditioned on Q , then the causal relations among P , Q and R must be one of the following types: $P \leftarrow Q \rightarrow R$, $P \rightarrow Q \rightarrow R$ or $P \leftarrow Q \leftarrow R$. Given three variables P , Q and R , Silverstein et al. [38] search for one independent pair (Q , R) and two correlated pairs (P , Q) and (P , R). If indeed Q is independent of R given P , then causal relationships $Q \rightarrow P \leftarrow R$ are established. Such methods are effective in finding specific structure locally, but may not be useful to discover the causal relationships over all the variables.

With an eye to improving efficiency, several methods [26] focused on discovering causal relationships between all pairs of variables using Markov blanket of a target variable. The Markov blanket of node T is a set $MB(T)$ where T is independent of other nodes conditioned on $MB(T)$. These methods have two phases: in the grow phase a Markov blanket or its superset is identified through a heuristic; in the shrink phase, the false positives are removed (Grow and Shrink [31], Incremental Association Markov Blanket [42]).

The divide-and-conquer approach (HITONPC/MB [2], MMPCMB [42], semi-Interleaved HITON-PC [2], PCMB [42] and IPC-MB [12]) find the immediate neighbours of the target node (parents and children) in the first step; subsequently, the spouses are identified. In these works, the task is to identify the parent–child set and Markov Blanket given a target variable. In our paper, we are focusing on discovering all the causal relationships. Hence, the tasks are different. Also, in our approach, there is no need to identify parent–child sets. Zhang et al. [45] uses a divide-and-conquer approach to find out partition of causal relationships given the graph with causal relations. In our case, we start with a graph with all possible edges (as the causal relationships among nodes are not known yet). Our aim is to discover the causal relationships and derive the graph from the node. Hence, the problem is different from that in [45].

Jin et al. [17] proposed the use of partial association and rule mining to discovery persistent relationship. Li et al. [22–24] are based on integration of concepts from control experiments and rule mining. Samples are divided into two groups such that they differ in some interests but share a common characteristic. Inferencing is done using the new dataset.

In [3], causal representation is learnt in weakly supervised manner using variable autoencoder (VAE). Causal representation learning has been leveraged in [27] for out-of-distribution generalization.

Efficient algorithmic techniques have been used in Zarebavani et al. [44] to parallelize CI tests. This helps in improving performance and handling extremely large networks using more GPUs. Scutari [37] leveraged parallel computing for causal discovery. Chen et al. [4] proposed a parallel learning algorithm for Bayesian network. Feng et al. [11] proposed a CPU-based parallel implementation of PC algorithm. The implementation reports significant speed-up due to savings in computation.

Parallel PC [21] is a parallel variant of the PC algorithm, with stability guarantee. Gao and Wei [13] follow a similar approach where agents learn local and global structures by message passing among the agents and using the exploration and exploitation strategy. This method has a high run time. The level of parallelization of both methods [13, 21] falls short for practical applications. We built a parallel recursive implementation that learns causal structures locally and uses them to build global causal relations. This has the advantage that if there is a sparse causal structure locally, the algorithm is able to leverage this. This is in contrast to the PC algorithm or other constraint-based global methods. These methods mostly start with a complete graph and fail at the initial stage for large instances. In the next subsection, we provide more details of closely related prior works.

2.1 The PC, STABLE-PC and Parallel-PC Algorithms

2.1.1 Definitions

Let $G = (V; E)$ be a graph with vertices V and edges $E \subseteq V \times V$. For any three vertices x, y and z , there are four possible types of indirect connections: causal path: $x \rightarrow z \rightarrow y$, evidential path: $x \leftarrow z \leftarrow y$, common cause $x \leftarrow z \rightarrow y$ and common effect $x \rightarrow z \leftarrow y$. The first three paths between x and y are active only when z is not observed, while for the common effect, the path is active only when z or any of its descendent is observed. The common effect $x \rightarrow z \leftarrow y$ structure is also called v-structure and z is called collider of the v-structure.

Let X, Y and Z are the three sets of nodes in G . X is d -separated from Y if no active path connects X and Y , given Z .

2.1.2 The PC algorithm

The PC algorithm [40] is a 2-step algorithm. It learns skeleton graph (undirected) from data in the first step. The undirected edges are oriented to get the final DAG in the second step.

The first step starts with a complete graph on all the variables. Afterwards, edges are removed based on d -separation of variables. This is based on the principle that if there is a subset of vertices Z from neighbourhood set of x and y such that it d -separates x and y , then there should not be an edge between nodes x and y . (In this case, x and y are independent conditioning on Z .)

Conditional independence tests discover a set Z that makes x and y conditionally independent. To identify Z , we may have to search all other variables, thus making the approach highly inefficient.

The algorithm constructs a clique and uses CI tests to prune edges. Advantage of considering an edge rather than any two vertices is that for identification of Z we need to check only the neighbours of x and y . In other words, the CI tests are carried out by levels(d). At $d = 0$ or the first level, all possible edges are tested. This may delete a few edges. In the next level ($d = 1$), the remaining edges are tested. After each iteration, the cardinality of the conditioning set d is incremented by one and the process is repeated until d exceeds the cardinality of the neighbours of the candidate pair of vertices. With each edge deletion, nodes will have modified neighbourhood. This is the key to its efficiency. When true relationships among variables are sparse,¹ fewer CI tests are required. Therefore, the algorithm will be fast for such instances.

¹ In a sparse network, the number of edges is of the linear order of number of nodes.

The algorithm's output depends on the order of edges (pair of adjacent vertices X and Y) considered. To alleviate this problem, Colombo et al. [6] proposed a modified version of the algorithm (called stable-PC algorithm). In the stable PC algorithm, at each level (d), all nodes explore their neighbours, but neighbours are not updated. The update happens only at the end of the processing at the level. As a result, the output graph is stable. Since there is no incremental pruning of neighbourhood of vertices, the algorithm needs more CI tests than the PC algorithm. This leads to increased runtime. An efficient and stable version of PC algorithm (the parallel-PC algorithm [21]) has been proposed to alleviate this.

2.1.3 Parallel-PC Algorithm

The parallel PC leverages parallel computation. In this algorithm, a task is broken into several independent smaller tasks, which are executed in parallel. The results from the smaller tasks are then integrated to construct the final solution. Nowadays, parallel computing is easily accessible, even on personal computers and laptops. Hence, the algorithm can be run easily.

Theoretically, the divide-and-conquer approach can improve speed almost equal to the number of cores. But this requires the smaller tasks to be unrelated. In other words, smaller tasks have to be independent with no mandatory communication among them. The stable-PC algorithm is slow due to additional CI tests. Only after completion of all the CI tests at a level, neighbours are updated.

The CI tests at each level are run in parallel in the parallel-PC algorithm. As the graph is updated before proceeding to the nodes at the next level, order-dependence is tackled properly. At a level, the CI tests are independent and hence can be run in parallel. It is possible to distribute the CI tests evenly across cores, thereby achieving the maximum speed-up.

In the parallel-PC algorithm, CI tests of the same edge are carried out on the same core to save on computation for the following cases (Le et al. [21]): Given an edge between x and y , it is expected that two CI tests are performed—one conditioning on the variables in $adj(x, G)$ and another in $adj(y, G)$. If after conditioning on $adj(x, G)$, the edge between x and y is removed, then the CI tests conditioning on $adj(y, G)$ are unnecessary.

3 The RPCD Algorithm

We are making the assumption that the input data is discrete, but we do not make any assumptions about the distribution. Conditional tests used as subroutines in the RPCD algorithm are G-square tests.

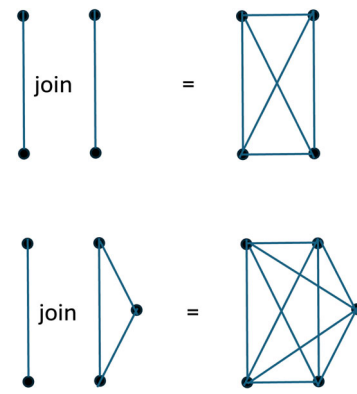


Fig. 1 Examples of complete joins

The parallel-PC algorithm constructs a clique of all nodes. However, the RPCD adopts a divide-and-conquer approach. Here, for each subsystem a clique is formed and the causal structure is learned. Learning can happen in parallel for different subsystems. Subsequently, the subsystems are merged pairwise. The steps are continued until the structure for the whole system is derived. As a result, the RPCD algorithm is much faster than the parallel-PC algorithm. RPCD is stable; the output does not depend on the order of evaluation of variable pairs in a dataset. Unlike the parallel-PC approach, the RPCD algorithm performs causal discovery on different partitions of variables in parallel and reconciles the results afterwards.

The RPCD algorithm is presented below (Algorithm 1). It has two steps. In the first step (skeleton discovery), the variables are partitioned into k subsets and cliques are constructed for each subset (Let these be $G_1^0, G_2^0, \dots, G_k^0$).² Parallel PC is run on each G_i^0 . As a result, skeletons g_c , $c \in \{1 \dots k\}$ are obtained. Pairs are formed from these skeleton graphs (there will be a singleton in case k is odd). For each pair (say g_c and g_d), we build a complete join g_{cd} (a complete join of two graphs $G(V_s, E_s)$ and $G(V_t, E_t)$ is $G(V_s \cup V_t, E_s \cup E_t \cup \{(a, b) : a \in V_s, b \in V_t\})$). Figure 1 illustrates two joins: one between a pair of path graphs and another between a path graph and triangle. Subsequently, parallel PC is run on each such join (e.g. g_{cd}) to discover skeleton. At the end of this phase, a set of skeletons (say G_c^1 , $c \in \{1 \dots q\}$) are formed. The second iteration starts with these skeletons. The process of merging and skeleton discovery is iterated till a single skeleton graph (G_{skel}) is obtained.

In the second step (edge orientation and cycle removal), we apply CPDAG to orient the edges. If there is a cycle, it is detected and removed in this step. Applying CPDAG can lead to cycles in the graph, so we need to devise a way to

² The order and size of partitioning are important. If the first partition corresponds to the sparsest cut, the speed-up is highest. But finding the sparsest balanced cut is computationally expensive. We used random partitioning.

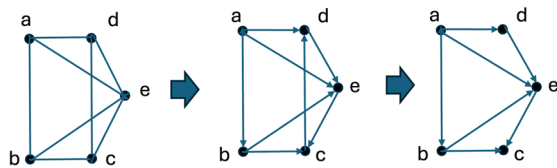


Fig. 2 Examples of cycle detection and removal-I

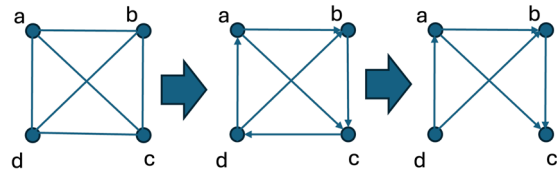


Fig. 3 Examples of cycle detection and removal-II

remove the cyclic edges. Heuristics to select the edge for removal depend on domain knowledge. In Fig. 2, left graph represents skeleton. Edge orientation results in the middle graph in which there is a cycle cde . By removing edge cd , finally we get a causal graph with no cycle. Often there can be multiple possibilities for the edge to be removed, and this can be done intelligently. For example in Fig. 3 (middle graph), there are multiple cycles ($abcd$, bcd and acd). Between the two smaller cycles, cd is the common edge. Hence, cd is a good choice for removal. By removing edge cd , finally we get a causal graph with no cycle.

Algorithm 1 RPCD Algorithm(Step 1): Skeleton Discovery

Input: Dataset D , significant level σ

Output: Undirected graph G and edgeset E

```

1: Partition the nodes into  $k$  sets. Construct  $k$  complete graphs
   ( $G_1^l, \dots, G_k^l$ ) one for each partition
2:  $S = \{G_1^l, \dots, G_k^l\}$ 
3: repeat
4:    $S' = \phi$ 
5:   for all graphs  $G_i \in S$  do
6:     Run parallel PC on  $G_i$ . Let the new skeleton graphs be  $g_i$ .
7:   end for
8:   for all pairs (say  $g_i$  and  $g_j$ ) do
9:     Build a complete join  $g_{ij}$ 
10:    Run parallel PC on  $g_{ij}$ . Let the new graph be  $H$ .  $S' = S' \cup \{H\}$ .
11:  end for
12:   $S = S'$ 
13: until there is only one component (i.e.,  $k = 1$ )

```

RPCD is analogous to depth-first traversal. It splits the nodes into two mutually exclusive subsets in layer 0. To construct a graph for a child node, it again splits the node. Splitting is top-down. Merging happens in a bottom-up manner, where two child graphs are merged into a single graph. It is to be noted that recursion and decomposition are important differentiators here. Due to these, the number of CI tests is much less than PC-stable (without compromising on stabil-

ity). RPCD does not have I/O overhead, as the input data is read only once. Finally, RPCD creates a task for learning the graph structure of a subgraph, and there is a separate 'merge' task for aggregating a pair of sub-tasks. There are many tasks that can run in parallel. For parallelism, we used pthreads. The following holds true for RPCD.

Theorem 1 *The worst-case run-time complexity of RPCD is $O(n^2 \log n)$, where n is the number of nodes.*

Proof sketch: Length of longest path from the root node to a leaf node is $O(\log n)$. Worst-case computation at each node is at most $O(n^2)$. Amortized complexity is expected to be better. $\square$

Theorem 2 *The graph resulting from RPCD at the end of skeleton discovery is correct in the sense it does not remove any true edge.*

Proof sketch: RPCD works in admissible manner. It does not remove any edge unless the conditional independence holds. If the conditional independence holds for the two nodes (a, b) of an edge ($e = ab$) with a neighbourhood set $N^g(a) \cup N^g(b)$ for a smaller graph g , it will hold for superset graph $h \supset g$, i.e. with $N^h(a) \cup N^h(b) \supseteq N^g(a) \cup N^g(b)$. Therefore, no true edge is eliminated by RPCD, and hence, correctness of RPCD holds true. $\square$

There are a number of trade-offs between breadth-first and depth-first with respect to memory overhead. Breadth-first offers more parallelism, but memory overhead is too much beyond a level and hence not feasible for practical instances (particularly for large-size instances). On the other hand, depth-first can use the same memory for storing the states for all nodes in its path from the root. This leads to significant memory efficiency. Assuming the state-space for a problem P has branching factor b and depth of $d - 1$, the following theorems capture the trade-off between depth-first and breadth-first approach.

Theorem 3 *The average and worst-case space and time complexity of breadth-first search for P is $O(b^d)$.*

Proof sketch: Space complexity is obvious as there are $O(b^d)$ nodes for the underlying graph for problem P . Worst-case time complexity will be $O(b^d)$ as in the worst case it requires solving sub-problems corresponding to each node in the graph. Average time complexity is the $O(E[b^{d-1} + (b^{d-1} - 1) + \dots + b + 1] = O(b^d)$. $\square$

Theorem 4 *The worst-case space and time complexity of depth-first search for P are $O(d)$ and $O(b^d)$, respectively.*

Proof sketch: Since depth-first algorithm needs to maintain a path from the root node to the current node, and this path is of maximum length d , space complexity is $O(d)$. The same

Table 1 Comparison of parallel-PC and RPCD on incident dataset

Number of samples	Number of events	Parallel-PC	RPCD
		Time taken(min)	Time taken(min)
500	1865	48	9
750	1865	99	11
1000	1865	212	15
1250	1865	251	18
1440	1865	—	21

space can be reused for other paths. In the worst case, all the sub-problems of the underlying graph need to be explored and solved; therefore, the time complexity is $O(b^d)$. $\square$

Theorem 5 *In the breadth-first algorithm, there can be $O(b^x)$ parallel workers at depth x , while for the depth-first algorithm it is $O(1)$.*

Proof sketch: Since each node corresponding to a sub-problem is stored in memory, at a level all the sub-problems can be solved in parallel. For the depth-first algorithm, a sub-problem corresponding to a node not on the current path cannot be solved until the traversal happens for the path. Branching can be better for a hybrid approach where additional nodes are used for storing nodes. $\square$

4 Empirical Results

We evaluated the RPCD algorithm in two domains: gene expression and software incidents. The gene expression dataset DREAM5-insilico [30] contains regulatory relationships between transcription factors (a type of gene regulator) and the regulated target genes. The DREAM5 network challenge consists of the genome-wide transcriptional data on the regulatory networks of *E. coli*, *S. cerevisiae* and *S. aureus* as well as an in silico (simulated) network. In gene networks, genes are represented as nodes and their relationships as edges.

We evaluated the performance of RPCD on a number of benchmark causal datasets (see Table 4). These are causal datasets with the number of nodes ranging from 50 to 250. More details of these datasets can be seen at <https://www.discretionary-bnlearn.com/discretionary-bnrepository/>. We used these datasets because these networks are large and have ground-truth data, so the accuracy of RPCD can be validated.

We also experimented with anomaly incidents from various subsystems in real-world systems. Here variables represent log events (e.g. request timed out, server down). RPCD was run on daily instances; there were 2000+ average log events daily. The events were captured at minute-level granularity. This translates to 1440 rows (or samples) for the

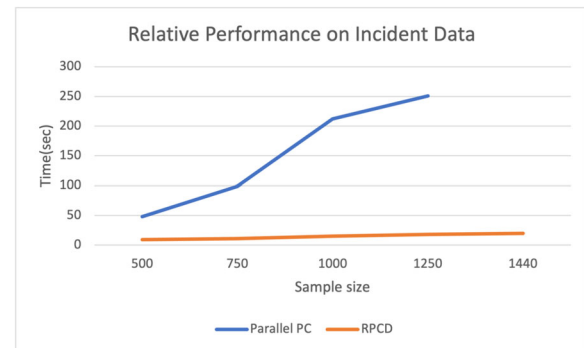


Fig. 4 Relative performance of RPCD and parallel PC on incident dataset

instance. The log events were partitioned based on the source (e.g. Kafka, Cassandra and so on).

During edge orientation through CPDAG, we can introduce cycles. When such a cycle arises, for each node in the cycle we trace the corresponding system. Since these are part of a pipeline, we know the order of execution (i.e. system flow) of these systems. (For example, we know that the offer-store depends on the Cassandra; hence, in the system flow there are edges from Cassandra to offer-store.) As a result, an edge in the cycle that defies the system flow can be removed. (Therefore, if we find an edge is directed from the Cassandra to the offer-store, it can be removed.)

Experiments were run on a Linux server (64-bit architecture) with 32 cores and 256GB RAM. Each core had an Intel processor (Intel(R) Xeon(R) Platinum 8171 M CPU, 2.60GHz, lithography 14nm, cache configurations: L1d cache - 32K, L1i cache- 32K, L2 cache - 1024K and L3 cache - 36608K). The system had a support of 16 cores per socket and 2 threads per core. For the experiments, we restricted the cores to 16, 20 and 24.

4.1 Runtime Efficiency

Table 1 portrays the runtime efficiency on incident databases. To compare the performance of the parallel-PC algorithm³ and the RPCD algorithm, we used the same datasets but var-

³ <https://cran.r-project.org/package=ParallelPC>

Table 2 Parallel-PC and RPCD against sample size (gene dataset)

Number of samples	Number of events	Parallel-PC	RPCD
		Time taken (min)	Time taken(min)
200	1643	133	1322
400	1643	707	1219
600	1643	1879	1294
805	1643	> 86400	2200

Table 3 Parallel-PC and RPCD against number of genes

Number of samples	Number of events	Parallel-PC	RPCD
		Time taken (min)	Time taken(min)
805	500	144	60
805	750	389	100
805	1000	909	240
805	1500	2848	900
805	1643	> 86400	2200

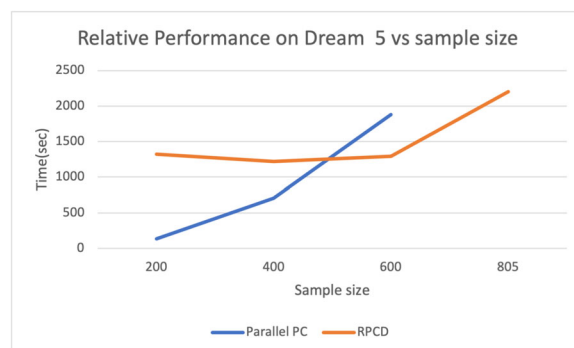
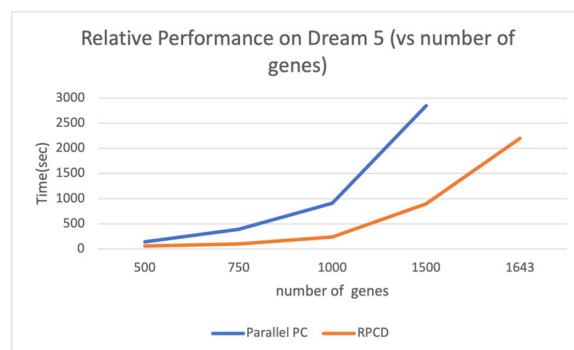
ied the number of variables. We used 16 cores to run both the parallel-PC and the RPCD algorithm for all the datasets. Table 1 shows that the runtime (elapse time) of the parallel-PC algorithm increases significantly with sample size. For 1440 samples of 1865 log events, the parallel-PC algorithm could not be completed even after 24 h; the RPCD algorithm took only 21 min. Figure 4 depicts that the relative run-time performance of the RPCD algorithm is improving with problem size.

Performance on Dream5 Gene database is given in Tables 2 and 3. All the runs were taken using 16 cores. Table 2 shows how the runtime increases with increase in samples. Though the Parallel-PC is faster on smaller instances (this can be attributed to extra overhead cost for RPCD), RPCD takes much less time on larger instances (Figure 5).

Table 3 depicts run-time performance with an increase in the number of genes. Figure 6 shows that run time increases significantly with the number of genes. This is due to the exponential increase of CI tests with the number of variables. However, the RPCD algorithm takes less time than parallel PC, and this improves with increasing data. To summarize, the RPCD algorithm out-performs the parallel-PC algorithm. In fact, for the largest instance, the parallel PC could not be completed within 24 h.

Figure 7 illustrates the relative performance of the two algorithms against availability of cores on the Dream5 dataset with 805 samples and 1643 genes. The RPCD algorithm is two times faster than the parallel PC. Time taken for RPCD decreases monotonically when additional cores are available, while there is no significant speed-up of the parallel-PC algorithm beyond 16 cores (also observed by other research).⁴

⁴ The difference is expected to be significant for a higher number of cores, but we are constrained by available cores. On the other hand,

**Fig. 5** Relative performance of RPCD and parallel PC against sample size on gene dataset**Fig. 6** Relative performance of RPCD and parallel PC against number of genes

Besides improvements in run time, RPCD has better performance than Parallel-PC. Average arrowhead precision improvement on the gene dataset is approximately 70% over

we could not make a comparison with a lower number of cores, as the parallel PC takes days to complete.

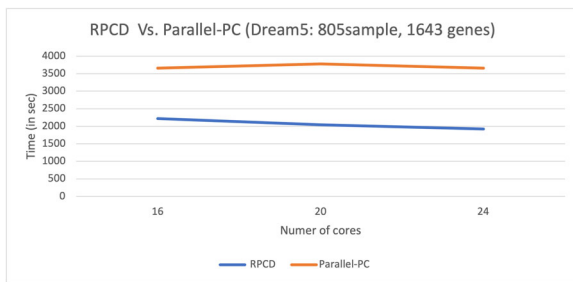


Fig. 7 Relative performance of RPCD and parallel PC Vs. cores

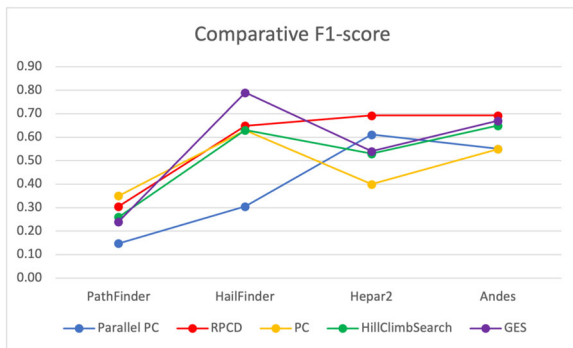


Fig. 8 Relative Performance on Benchmark data

that of Parallel-PC.⁵ Arrowhead recall for RPCD is 10 times that of parallel PC. For the largest gene network, the arrowhead F1-score for RPCD is 0.036, while that of parallel PC is 0.0039. The results establish that the RPCD algorithm improves upon both speed and accuracy. The improvement of speed is significant.

RPCD performance is significantly better than parallel PC on most of the benchmark dataset. Figure 8 shows that RPCD has better performance than other methods.⁶ Accuracy improvement ranges from 20% to 125%. Interestingly, run time is improved too. As can be seen from Figure 9, RPCD is 2–90 times faster on these instances. Figure 9 shows that optimized methods like HillClimbSearch and GES have significantly higher run time. (In fact for Andes dataset these took 323.57 min and 214.2 min, respectively.)

We have experimented with different values of number of partitions (k). With higher value of k , the execution time increases (Table 5). Hence, we decided to continue further experiment with $k = 2$ only. We observed similar behaviour with smaller sample size and variables. As observed in [21], run time increases significantly when the number of variables as the number of CI tests is more. Similarly, run time of RPCD goes up with increase in the number of samples. However, the reason for this is increased execution time taken to perform

⁵ Precision/recall improvement on incident database could not be done due to non-availability of ground-truth data.

⁶ GES performing better only on Hailfinder.

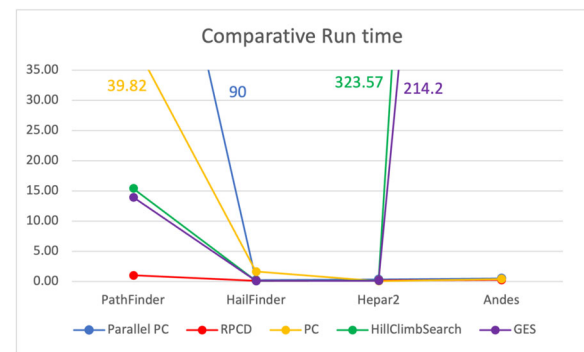


Fig. 9 Relative Execution Time on Benchmark data

an independence test. Also, there are more false edges added with higher value of k . Adjacency precision improves by 67% for binary split ($k = 2$) over quadruplet split ($k = 4$).

Accuracy of causal models depends on the chosen significance level (σ) of the CI tests. Most common value used is 0.05. A lower significance level (e.g. below 0.01) makes the test stringent, increasing the number of false negatives. On the other hand, a higher significance level (e.g. more than 0.10) leads to higher likelihood of rejecting the null hypothesis when it may actually be true. This increases the risk of false positives. Since our objective is to compare the relative performance of various methods, we used the same σ (0.05) across all the methods.

5 Concluding Remarks

Discovering regulatory relationships in the real world is a problem of significant practical interest. For instance, the discovered relationships from gene expression data can help establish the root causes of fatal diseases. Similarly, causal relationships can help predict system failures in manufacturing and other production systems. This has implications in terms of preventing production losses. However, most of the real-life instances are large and high-dimensional; existing algorithms fail for such instances. For obvious reasons, significant efforts have been made to improve the running time and scalability of causal discovery approaches. Nevertheless, these still fall short of expectations. For example, on Dream 5 datasets, we have to wait for 24 h. Our experiments on real-life software systems showed that it cannot be completed in such instances, even after a day of elapsed time.

The RPCD algorithm improves the state of the art. Experimental evidence suggests that the new approach is efficient and scalable to handle real-life instances from different domains. As a specific example, for a Dream 5 instance with 1643 genes, the RPCD algorithm took only 36 min (the parallel PC took more than 24 h). Besides, RPCD has better accuracy than the parallel PC. The key factors for the

Table 4 Accuracy of parallel-PC and RPCD on Benchmark datasets

Dataset	Nodes	Arcs	Parallel-PC	RPCD	Accuracy Improvement(%)
			F1-score	F1-score	
Pathfinder	90	1	0.15	0.31	125
Hailfinder	0.2	0.1	0.55	0.69	167
Hepar2	0.3	0.16	0.61	0.69	20
Andes	0.5	0.25	0.31	0.65	40

Table 5 Sensitivity on number of Splits (k) on Dreams

Number of samples	Number of variables	Number of splits(k)	Time Taken (min)
805	1643	2	20
		4	37

improvement are recursion and decomposition. These have combined effect of reducing the number of CI tests, thereby improving the speed. Usage of depth-first strategy helps in significant memory efficiency.

We tried to solve the scalability problem through problem decomposition. It would be worthwhile to try other orthogonal approaches. Another fruitful research direction could be the design of suitable heuristics that yield good approximate solutions but have improved runtime.

One limitation of RPCD is on minimum memory requirement. Since it is based on depth-first strategy, for successful operation it requires that there is sufficient memory to store nodes on path from the root node to a leaf node. (In other words, minimum memory requirement is proportional to the depth of tree at each stage.) Another limitation of RPCD is the usage of rudimentary parallelism based on Pthread. Advanced parallelization schemes can be used to improve RPCD further.

Author Contributions Sakib conceptualized and designed the algorithm and developed prototype, planned and reviewed all the work contained in this article. All authors contributed to the implementation, with Sagar doing most of it. Prashanth worked on datasets and experimental runs. All authors reviewed the manuscript.

Funding There is no funding for this work.

Data Availability DREAMS5 Dataset is available from <https://www.synapse.org/Synapse:syn2787209/files>. Data on other Bayesian networks are available from <https://www.bnlearn.com/bnrepository>. Incident data contains confidential information about the organization and hence cannot be shared openly.

Declarations

Conflict of interest There is no conflict of interest.

References

1. Abellán, Joaquín, Gómez-Olmedo, Manuel, Moral, Serafín, et al.: Some variations on the PC algorithm. In *Probabilistic graphical models*, 1–8. Citeseer, (2006)
2. Aliferis, Constantin F., Statnikov, Alexander, Tsamardinos, Ioannis, Mani, Subramani, Koutsoukos, Xenofon D.: Local causal and Markov blanket induction for causal discovery and feature selection for classification part i: algorithms and empirical evaluation. *Journal of Machine Learning Research*, **11**(1), (2010)
3. Brehmer, Johann, De Haan, Pim, Lippe, Phillip, Cohen, Taco S.: Weakly supervised causal representation learning. *Advances in Neural Information Processing Systems* **35**, 38319–38331 (2022)
4. Chen, Yetian, Tian, Jin, Nikolova, Olga, Aluru, Srinivas: A parallel algorithm for exact Bayesian structure discovery in bayesian networks. arXiv preprint [arXiv:1408.1664](https://arxiv.org/abs/1408.1664), (2014)
5. Chickering, David Maxwell: Learning Bayesian networks is NP-complete. *Learning from data: Artificial intelligence and statistics V*, 121–130, (1996)
6. Colombo, Diego, Maathuis, Marloes H., Kalisch, Markus, Richardson, Thomas S.: Learning high-dimensional directed acyclic graphs with latent and selection variables. *The Annals of Statistics*, 294–321, (2012)
7. Cooper, Gregory F.: A simple constraint-based algorithm for efficiently mining observational databases for causal relationships. *Data Mining and Knowledge Discovery* **1**, 203–224 (1997)
8. Cooper, Gregory F., Herskovits, Edward: A Bayesian method for the induction of probabilistic networks from data. *Machine learning* **9**, 309–347 (1992)
9. Dash, Denver, Druzdzel, Marek J.: A hybrid anytime algorithm for the construction of causal models from sparse data. arXiv preprint [arXiv:1301.6689](https://arxiv.org/abs/1301.6689), (2013)
10. Edwards, David: Introduction to graphical modelling. Springer Science & Business Media (2000)
11. Feng, Yongquan, Xu, Liyang, Huang, Wanrong, Yang, Wenjing, Lu, Yutong: PEPC: Parallel and Extensible PC implementation for causal structure learning. In *Proceedings of the 2023 7th International Conference on High Performance Compilation, Computing and Communications*, 77–84, (2023)
12. Fu, Shunkai, Desmarais, Michel C.: Fast markov blanket discovery algorithm via local learning within single pass. In *Advances in Artificial Intelligence: 21st Conference of the Canadian Society for Computational Studies of Intelligence, Canadian AI 2008 Windsor, Canada, May 28–30, 2008 Proceedings 21*, 96–107. Springer, (2008)

13. Gao, Tian, Wei, Dennis: Parallel Bayesian network structure learning. In *International Conference on Machine Learning*, pages 1685–1694. PMLR, (2018)
14. Glymour, Clark N., Cooper, Gregory Floyd: Computation, causation, and discovery. (*No Title*), (1999)
15. Granger, C.W.J.: Investigating causal relations by econometric models and cross-spectral methods. In *Essays in econometrics: collected papers of Clive WJ Granger*, 31–47. (2001)
16. Heckerman, David, Geiger, Dan, Chickering, David M.: Learning Bayesian networks: The combination of knowledge and statistical data. *Machine learning* **20**, 197–243 (1995)
17. Jin, Zhou, Li, Jiuyong, Liu, Lin, Le, Thuc Duy, Sun, Bingyu, Wang, Rujing: Discovery of causal rules using partial association. In *2012 IEEE 12th International Conference on Data Mining*, 309–318. IEEE, (2012)
18. Kalisch, Markus, Bühlman, Peter: Estimating high-dimensional directed acyclic graphs with the PC-algorithm. *Journal of Machine Learning Research*, **8**(3), (2007)
19. Kalisch, Markus, Mächler, Martin, Colombo, Diego, Maathuis, Marloes H., Bühlmann, Peter: Causal inference using graphical models with the R package pcalg. *Journal of statistical software* **47**, 1–26 (2012)
20. Le, Thuc Duy, Liu, Lin, Tsykin, Anna, Goodall, Gregory J., Liu, Bing, Sun, Bing-Yu., Li, Jiuyong: Inferring microRNA-mRNA causal regulatory relationships from expression data. *Bioinformatics* **29**(6), 765–771 (2013)
21. Le, Thuc Duy, Hoang, Tao, Li, Jiuyong, Liu, Lin, Liu, Huawen, Shu, Hu.: A fast PC algorithm for high dimensional causal discovery with multi-core pcs. *IEEE/ACM transactions on computational biology and bioinformatics* **16**(5), 1483–1495 (2016)
22. Li, Jiuyong, Le, Thuc Duy, Liu, Lin, Liu, Jixue, Jin, Zhou, Sun, Bingyu: Mining causal association rules. In *2013 IEEE 13th International Conference on Data Mining Workshops*, 114–123. IEEE, (2013)
23. Li, Jiuyong, Le, Thuc Duy, Liu, Lin, Liu, Jixue, Jin, Zhou, Sun, Bingyu, Ma, Saisai: From observational studies to causal rule mining. *ACM Transactions on Intelligent Systems and Technology (TIST)* **7**(2), 1–27 (2015)
24. Li, Jiuyong, Liu, Lin, Le, Thuc Duy: *Practical approaches to causal relationship exploration*. Springer, (2015)
25. Ling, Zhaolong, Li, Ying, Zhang, Yiwen, Kui, Yu., Zhou, Peng, Li, Bo., Xindong, Wu.: A light causal feature selection approach to high-dimensional data. *IEEE Transactions on Knowledge and Data Engineering* **35**(8), 7639–7650 (2022)
26. Ling, Zhaolong, Kui, Yu., Zhang, Yiwen, Liu, Lin, Li, Jiuyong: Causal learner: A toolbox for causal structure and markov blanket learning. *Pattern Recognition Letters* **163**, 92–95 (2022)
27. Lu, Chaochao, Wu, Yuhuai, Hernández-Lobato, José Miguel, Schölkopf, Bernhard: Invariant causal representation learning for out-of-distribution generalization. In *International Conference on Learning Representations*, (2021)
28. Maathuis, Marloes H., Colombo, Diego, Kalisch, Markus, Bühlmann, Peter: Predicting causal effects in large-scale systems from observational data. *Nature methods* **7**(4), 247–248 (2010)
29. Madsen, Anders L., Jensen, Frank, Salmerón, Antonio, Langseth, Helge, Nielsen, Thomas D.: A parallel algorithm for Bayesian network structure learning from large data sets. *Knowledge-Based Systems* **117**, 46–55 (2017)
30. Marbach, Daniel, Costello, James C., Küffner, Robert, Vega, Nicole M., Prill, Robert J., Camacho, Diogo M., Allison, Kyle R., Kellis, Manolis, Collins, James J., et al.: Wisdom of crowds for robust gene network inference. *Nature methods* **9**(8), 796–804 (2012)
31. Margaritis, Dimitris, Thrun, Sebastian: Bayesian network induction via local neighborhoods. *Advances in neural information processing systems*, **12**, (1999)
32. Neapolitan, Richard E., et al. *Learning bayesian networks*, volume 38. Pearson Prentice Hall Upper Saddle River, (2004)
33. Pearl, Judea: *Probabilistic reasoning in intelligent systems: networks of plausible inference*. Morgan kaufmann, (1988)
34. Pearl, Judea: Introduction to probabilities, graphs, and causal models. *Causality: models, reasoning and inference*, 1–40, (2000)
35. Robinson, Robert W.: Counting labeled acyclic digraphs. *New directions in the theory of graphs*, 239–273, (1973)
36. Scutari, Marco: Learning Bayesian networks with the bnlearn R package. arXiv preprint [arXiv:0908.3817](https://arxiv.org/abs/0908.3817), (2009)
37. Scutari, Marco: Bayesian network constraint-based structure learning algorithms: Parallel and optimised implementations in the bnlearn R package. arXiv preprint [arXiv:1406.7648](https://arxiv.org/abs/1406.7648), (2014)
38. Silverstein, Craig, Brin, Sergey, Motwani, Rajeev, Ullman, Jeff: Scalable techniques for mining causal structures. *Data Mining and Knowledge Discovery* **4**, 163–192 (2000)
39. Sims, Christopher A.: Money, income, and causality. *The American economic review* **62**(4), 540–552 (1972)
40. Spirtes, Peter, Glymour, Clark, Scheines, Richard: Causation, prediction, and search. MIT press (2001)
41. Steck, Harald, Tresp, Volker: Bayesian belief networks for data mining. In *Proceedings of the 2. workshop on data mining und data warehousing Als Grundlage Moderner Entscheidungsunterstützender Systeme*, 145–154. Citeseer, (1999)
42. Tsamardinos, Ioannis, Aliferis, Constant F., Statnikov, Alexander R., Statnikov, Er: Algorithms for large scale Markov blanket discovery. In *FLAIRS conference*, **2**, 376–380. St. Augustine, FL, (2003)
43. Verma, Thomas S., Pearl, Judea: Equivalence and synthesis of causal models. In *Probabilistic and causal inference: The works of Judea Pearl*, 221–236. (2022)
44. Zarebavani, Behrooz, Jafarinejad, Foad, Hashemi, Matin, Salehkalebar, Saber: cuPC: CUDA-based parallel PC algorithm for causal structure learning on GPU. *IEEE Transactions on Parallel and Distributed Systems* **31**(3), 530–542 (2019)
45. Zhang, Junpeng, Le, Thuc Duy, Liu, Lin, Liu, Bing, He, Jianfeng, Goodall, Gregory J., Li, Jiuyong: Identifying direct mirna-mrna causal regulatory relationships in heterogeneous data. *Journal of biomedical informatics* **52**, 438–447 (2014)
46. Zhang, Junpeng, Le, Thuc Duy, Liu, Lin, Liu, Bing, He, Jianfeng, Goodall, Gregory J., Li, Jiuyong: Inferring condition-specific mirna activity from matched mirna and mrna expression data. *Bioinformatics* **30**(21), 3070–3077 (2014)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.